GUIA DE LABORATORIO 1

CURSO:	Lenguajes de programación
DOCENTE:	Lenin Froy Arce
FECHA:	21/08/2026
TEMA:	Lenguajes de programación y paradigmas
ESTUDIANTES:	Percy José Ortega Sandoval Reena Chadni Arroyo Quispe Camila Yamilet Moncada Llocclla kilyan Valentino Montoya Alvan

1.Código en paradigmas

- Paradigma estructurado

```java
package a1;

import java.util.Scanner;

public class C1 {
    public static void main(String[] args) {
        Scanner entrada =new Scanner(System.in);

        Double A1,A2,R;
        System.out.println("ingrese el primer numero a sumar");
        A1=entrada.nextDouble();
        System.out.println("ingrese el segundo numero a sumar");
        A2=entrada.nextDouble();
        R=A1+A2;
        System.out.println("el resultado es "+R);
        entrada.close();
```

Problems Javadoc Declaration Console Eclipse IDE for Java Developers 2026-09 M3

<terminated> C1 [Java Application] C:\Users\percy\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32

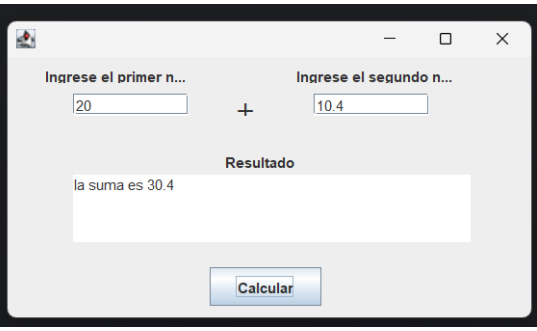
```
ingrese el primer numero a sumar
20
ingrese el segundo numero a sumar
10.4
el resultado es 30.4
```

- Paradigma orientado a eventos

```
    }

}
public void actionPerformed(ActionEvent e) {
    if (e.getSource() == btnNewButton) {
        do_btnNewButton_actionPerformed(e);
    }
}
protected void do_btnNewButton_actionPerformed(ActionEvent e) {
    double N1=Double.parseDouble(txt1.getText());
    double N2=Double.parseDouble(txt2.getText());
    double SU=N1+N2;
    txtR.append(("la suma es "+SU));

}
```



- Paradigma orientado a objetos (OOP)

```
package a1;

public class calcular {

        double num1,num2;

        double sumar() {
            return num1 + num2;
```

```
4
5  public static void main(String[] args) {
6    Scanner entrada=new Scanner(System.in);
7    calcular suma=new calcular();
8    System.out.println("Ingrese el primer numero");
9    suma.num1=entrada.nextDouble();
10   System.out.println("ingrese el segundo numero");
11   suma.num2=entrada.nextDouble();
12   double resultado=suma.sumar();
13   System.out.println("el resultado es: "+resultado);
14
15 }
16
17 }
```

```
Problems  Javadoc  Declaration  Console  Eclipse IDE for Java De
<terminated> POO1 [Java Application] C:\Users\percy\.p2\pool\plugins\org.eclipse.justj.op
Ingrese el primer numero
15
ingrese el segundo numero
20.4
el resultado es: 35.4
```

- Paradigma funcional

```java
package a1;
import java.util.Scanner;

public class F1 {


    static final BinaryOperator<Double> suma = (num1, num2) -> num1 + num2;

    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);

        double num1 = leer(entrada, "Ingrese el primer número: ");
        double num2 = leer(entrada, "Ingrese el segundo número: ");

        double resultado = suma.apply(num1, num2);

        Resultado(resultado);

        entrada.close();
    }

       static double leer(Scanner entrada, String mensaje) {
        System.out.print(mensaje);
        return entrada.nextDouble();
    }

    static void Resultado(double resultado) {
        System.out.println("La suma es: " + resultado);
    }
}
```

```
Problems  Javadoc  Declaration  Console  Eclipse IDE for Java Developers 2026-09 M3
<terminated> F1 [Java Application] C:\Users\percy\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.12.v202
Ingrese el primer número: 10
Ingrese el segundo número: 20
La suma es: 30.0
```

2. Gramáticas escritas en BNF/EBNF

BNF

<expresión> ::= <numero> "+" <numero>

<numero>:: = <digito> | <numero> <digito>

<digito>:: = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

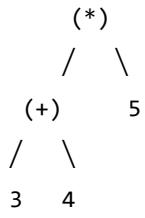
EBNF

<expresión> = <numero> "+" <numero> ;

<numero> = <digito> { <digito> } ;

<digito> = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" ;

3. Árbol de sintaxis abstracta dibujado para la expresión (3 + 4) * 5



4. Código generado con JLex y JCup, y ejecución de pruebas.

Jlex:

v1.java | SumaEstructurada.java | Lexer.flex | Parser.cup | Main.java

```
1 import java_cup.runtime.Symbol;
2
3 %%
4
5 %class Lexer
6 %cup
7
8 %%
9
10 [0-9]+  { return new Symbol(sym.NUMERO); }
11 "+"     { return new Symbol(sym.MAS); }
12 "-"     { return new Symbol(sym.MENOS); }
13 "*"     { return new Symbol(sym.POR); }
14 "/"     { return new Symbol(sym.DIV); }
15
16 [ \t\r\n]+   { }
17
18 . { System.out.println("Símbolo no reconocido: " + yytext()); }
```

Problems | Javadoc | Declaration | Console | Eclipse IDE for Java Developers 2026-09 M3 | Sponsor the Eclipse IDE

```
<terminated> Main [Java Application] C:\Users\User\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.12.v202608
Expresion valida
```

JCup

v1.java | SumaEstructurada.java | Lexer.flex | Parser.cup | Main.java

```
terminal NUMERO, MAS, MENOS, POR, DIV;

non terminal expresion;

precedence left MAS, MENOS;
precedence left POR, DIV;

start with expresion;

expresion ::= expresion MAS expresion
            | expresion MENOS expresion
            | expresion POR expresion
            | expresion DIV expresion
            | NUMERO
            ;
```

Problems | Javadoc | Declaration | Console | Eclipse IDE for Java Developers 2026-09 M3 | Sponsor th

<terminated> Main [Java Application] C:\Users\User\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_2

```
Expresion valida
```

v1.java | SumaEstructurada.java | Lexer.flex | Parser.cup | Main.java

```java
import java.io.StringReader;

public class Main {

    public static void main(String[] args) {

        try {
            String expresion = "10 + 20";

            Lexer lexer = new Lexer(new StringReader(expresion));
            Parser parser = new Parser(lexer);

            parser.parse();

            System.out.println("Expresion valida");

        } catch (Exception e) {
            System.out.println("Expresion no valida");
        }
    }
}
```

Problems | Javadoc | Declaration | Console | Eclipse IDE for Java Developers 2026-09 M3

<terminated> Main [Java Application] C:\Users\User\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.

```
Expresion valida
```